AvoTalent — Guía de implementación, semana 1

Para Francisco · semana del 16 al 22 de noviembre de 2026 · tareas 21, 22, 56, 32 y 45 · preparada el 13 de noviembre de 2026

Qué es este documento. El PDF de diseño funcional y técnico del jefe de producto decía «no tengo acceso al código: los nombres son sugerencias». Esta guía lo completa: contesta **todas las preguntas abiertas** con lo que realmente hay en el repositorio y en la base (marcado HECHO), corrige los nombres a los que ya existen, y da **los pasos exactos** de cada tarea (archivos, SQL, código de referencia, pruebas y cierre) para que se ejecute como una receta. Lo que no se pudo comprobar está marcado VERIFICAR; las decisiones que son del fundador, DECISIÓN.

0. Cómo se trabaja en este repositorio (leer una vez)

- **Estructura.** `backend/` (Express + TypeScript, Railway), `apps/community/` (Next 16.3.3, App Router, Vercel), `apps/avocado/` (panel Telemetry, Next), `packages/mcp-analytics/` (el servidor del agente de producto). El frontend **nunca** habla con Supabase: todo pasa por el backend (`/api/community/*` público o con sesión, `/api/admin/analytics/*` solo superadmin).
- **No hay entorno de prueba (staging).** HECHO Un `git push` a `master` despliega solo en Vercel y Railway. Por eso: (1) todo comportamiento nuevo que toque a las personas o publique algo va detrás de un **interruptor de entorno apagado por defecto**; (2) se prueba local con `npx tsc --noEmit` y `npx vitest run` (backend ≈1456 pruebas, comunidad ≈694) antes de subir; (3) los scripts nuevos tienen modo de prueba que no escribe.
- **Migraciones SQL.** No hay corredor de migraciones: cada una es un archivo `backend/sql/<nombre>-migration.sql`, **idempotente** (`IF NOT EXISTS`), que el fundador pega y corre en el editor SQL de Supabase. Tabla nueva = `ENABLE ROW LEVEL SECURITY` + `REVOKE ALL ... FROM anon, authenticated`. El código debe **tolerar que la migración aún no se haya corrido** (responder `{ ready: false }`, nunca 500). Lección real de esta semana: se asumió una columna (`community_votes.created_at`) que no existía y la ruta respondió 500; antes de leer una columna nueva, verificarla.
- **Rutas del panel.** Todo bajo `backend/src/routes/admin/analytics.routes.ts` con el envoltorio `handle('nombre', async (req, res) => …)`. Para que el agente de producto lea una métrica: (a) agregar la ruta agregada (**sin correos ni nombres**) a `SERVICE_READABLE` en `backend/middleware/analytics-access.middleware.ts`; (b) agregar la entrada a `PANEL_METRICS` en `packages/mcp-analytics/src/panel.ts`. Una ruta que lleve personas **no** va en `SERVICE_READABLE` (hay una prueba que lo vigila: `tests/unit/analytics-access.test.ts`).
- **Pantallas del panel.** Páginas en `apps/avocado/src/app/(panel)/panel/<nombre>/page.tsx`; datos con `usePanelData<T>(ruta)` y envíos con `api<T>(ruta, { method, body })` (`src/lib/api.ts`); el menú está en `components/panel/PanelShell.tsx` (`NAV_GROUPS`); toda tabla usa `components/panel/DataTable.tsx`.
- **Anotar en `get_change_log`.** Cada commit `feat` / `fix` / `perf` lleva en el cuerpo una línea `Expects: <métrica y dirección>` y se registra solo al desplegar en Railway. Lo que no es un commit (correr una migración, prender un interruptor, aplicar datos) se anota a mano: `npm run log-change -- "título" --kind config --area x --expects "métrica" --at 2026-11-17T15:00:00Z` (la hora **en UTC con Z**; sin Z se interpreta en hora local y queda 6 horas corrida — error ya cometido una vez). Tipos válidos: `feature, fix, experiment, config, content, chore`.
- **Al terminar cada tarea** se agrega una viñeta al `CLAUDE.md` de la raíz (qué es, archivos, interruptores) y se devuelve lo que pide la sección final de este documento.

1. Lo que ya existe y el PDF original no sabía (tabla de correcciones)

El PDF dice / propone	La realidad en el código
«web_apply_funnel cuenta clics»	Ya cuenta **personas** por dispositivo × con/sin cuenta × paso (abrió, vio el botón, tocó, salió) con la función SQL `analytics_apply_funnel`, ruta `GET /api/admin/analytics/web/apply-funnel`. Lo que **no** tiene es «dijo que se postuló / todavía no»: eso vive en otra tabla (ver tarea 22).
«respuestas del punto 46»	Se guardan en la tabla `vacancy_applications` (una fila por persona + vacante, con `user_id`). Ver tarea 22.
Campos nuevos en vacantes	La tabla de vacantes publicadas es `community_posts` (`type = 'job'`). **No** existe ningún campo de estado del enlace ni fecha de revisión (verificado: la columna `link_status` no existe).
«propuestas en `linkedin_proposals`»	Hoy existe `linkedin_vacancy_posts` (qué se publicó, con `UNIQUE(vacancy_id)`). Las propuestas son una tabla nueva y aparte.
«API de Anthropic»	El backend **no tiene** el SDK de Anthropic; usa `@google/genai` (Gemini) para contenido y scraper. Hay que agregar `@anthropic-ai/sdk` y la variable `ANTHROPIC_API_KEY` (la pone el fundador en Railway).
«foto semanal de engagement_history»	Existe: tabla `engagement_snapshots` (tipos `semanal`, `base`, `reconstruida`), servicio `backend/services/analytics/engagement-history.ts`, ruta `/engagement-history/summary`, métrica del agente `engagement_history`. Primera foto semanal real: lunes 9 de nov (semana del 2 al 8).
«VALUE_POINTS, un solo lugar»	Es `backend/services/analytics/people-value.ts`: ahí están `VALUE_POINTS`, `TEMPERATURE`, `ENGAGEMENT_DAYS`, `ACTIVE_DAY_POINTS`, `MAX_PER_ACTION`.
«Open Graph»	Ya hay etiquetas en la vacante (`app/(main)/vacantes/[slug]/layout.tsx`) pero **incompletas**: usa el logo de la empresa como imagen (si existe) y `twitter:card = summary`. Ver tarea 32.

2. Decisiones que siguen siendo del fundador

Francisco **no** las decide. Cada una trae una recomendación y un valor por defecto para no quedar bloqueado: se implementa con el valor por defecto y se cambia con una variable de entorno.

#	Decisión	Recomendación / valor por defecto
D1	Tope de gasto diario de la API de Anthropic (tarea 56)	Variable `LINKEDIN_AGENT_DAILY_BUDGET_USD`, por defecto **1.00**. Una llamada al día con ≈10 000 tokens de entrada y ≈1 500 de salida cuesta centavos; el tope es una red de seguridad, no el gasto esperado. El precio exacto por millón de tokens se lee de la tabla vigente de Anthropic VERIFICAR y se guarda en constantes configurables.
D2	Qué pasa si el fundador no aprueba antes de la hora del post	**No se publica** ese post y se avisa en el panel (propuesta del jefe de producto). Alternativa no recomendada: publicar el respaldo sin aprobación, que rompe la regla «nada se publica sin aprobación».
D3	Logos de empresas en las tarjetas (tarea 32)	**Apagado** (`OG_SHOW_LOGOS=off`): insignia con la inicial. 685 de 905 vacantes abiertas tienen logo, pero se guardan copiados en nuestro almacenamiento y **no hay licencia verificada**. Hoy la tarjeta de vacante ya los usa como imagen: este cambio quita ese riesgo.
D4	El caso «dormida con temperatura tibia/caliente» (tarea 45)	Existe de verdad (1 cuenta hoy). Recomendación: asignarla al **conjunto 4 (Reactivadas)** y marcarla con `state_lag = true`. Explicación en la tarea 45.
D5	Capacidad de la semana	Ver sección 3. Si hay que mover una tarea, la 45.

3. Orden recomendado y capacidad (estimación)

ESTIMACIÓN (Francisco confirma): 21 ≈ 1.5 días, 22 ≈ 0.5–1 día, 32 ≈ 1.5 días, 56 ≈ 2 días, 45 ≈ 1 día. Suman ≈ 7 días con 5 disponibles: **no alcanzan las cinco con holgura**; la que se mueve a la semana 2 es la 45 (de acuerdo con el jefe de producto).

Cambio de orden importante: el 56 depende de la imagen del 32. Un post de LinkedIn se publica como **enlace con tarjeta** (`shareMediaCategory: 'ARTICLE'` en `services/linkedin.service.ts`): la imagen que se ve la **lee LinkedIn de las etiquetas Open Graph de la página**. Por eso «una sola plantilla de imagen» sale gratis: la plantilla OG del 32 **es** la imagen del post. No hay que subir imagen aparte ni usar Google Imágenes. Orden:

1. **Lun 16–mar 17:** 21 y 22 (independientes).
2. **Mié 18 (mañana):** solo la **ruta de imagen OG** del 32 (pasos 1–3 de esa tarea).
3. **Mié 18 (tarde)–jue 19:** 56 completa.
4. **Jue 19–vie 20:** resto del 32 (etiquetas en las 4 clases de página y verificación con los validadores).
5. **Vie 20 (solo si sobra):** 45. Si no, semana 2.

Muro de vacantes. HECHO Hoy el detalle de una vacante exige sesión (decisión del 1 nov): `GET /api/community/posts/:id` responde 401 a invitados; lo que ven invitados, buscadores y LinkedIn es la versión pública sin contacto `GET /api/community/seo/vacancy/:slug`. El 32 debe usar **solo** ese endpoint; así, aunque el fundador cambie el muro, la tarjeta no filtra nada.

Tarea 21 — Verificación de enlaces de postulación

Lun 16 – mar 17 nov · esfuerzo S–M · solo mide y marca; no oculta ni borra

RESPUESTAS A LAS PREGUNTAS ABIERTAS

- **¿Hay un campo de estado o fecha de revisión en la tabla de vacantes?** HECHO No. Hay que agregarlos a `community_posts`.
- **¿El scraper corre en el mismo servidor que la API?** HECHO Sí: un solo servicio en Railway; los crons del scraper (`initScraperCron`) y los de contenido arrancan dentro del mismo proceso de la API (`initCron()` en `backend/index.ts`). Consecuencia: el trabajo nuevo debe ser **liviano y con espera entre peticiones** (no cargar todo en memoria, lotes de 50, ≤5 peticiones por segundo) para no quitarle tiempo a las respuestas de la API.
- **¿Qué proveedores bloquean peticiones sin navegador?** HECHO **Las 905 vacantes abiertas vienen de Greenhouse** (plataforma `greenhouse`). Los enlaces son: `job-boards.greenhouse.io` (413), `boards.greenhouse.io` (120) y páginas de carrera propias de las empresas que usan Greenhouse por detrás (`stripe.com` 38, `app.careerpuck.com` 27, `instacart.careers` 25, `wizeline.ai` 23, `mongodb.com` 21, `coinbase.com` 21, …). No conocemos bloqueos por proveedor; por eso existe el estado «desconocido».
- **¿Dentro del scraper o como trabajo aparte? Aparte.** El scraper decide qué *entra*; esto revisa lo que *ya está abierto* y debe correr también entre lotes (el lote llega cada ~15 días, pero ninguna vacante puede quedar más de 7 días sin revisar). Va como módulo propio `backend/services/vacancies/link-check.ts` con su propio cron, sin tocar el flujo del scraper.

CÓMO SE REVISA (CLAVE DEL DISEÑO)

Como todas las vacantes son de Greenhouse, **no hace falta raspar páginas HTML ni pelear con bloqueos**: Greenhouse tiene una API pública documentada, y el scraper ya la usa (`https://boards-api.greenhouse.io/v1/boards/{board}/jobs?content=true`). La revisión por vacante es:

```
GET https://boards-api.greenhouse.io/v1/boards/{board}/jobs/{jobId}
  200  -> la oferta sigue abierta        -> 'ok'
  404  -> la oferta ya no existe/cerró    -> 'roto' (motivo 'oferta_cerrada')
```

- `{board}` = `community_posts.source_name` (el token del tablero, p. ej. `stripe`). `{jobId}` sale de `source_url` con la expresión `/(?:gh_jid=|\/jobs\/)(\d+)/` (sirve para `job-boards.greenhouse.io/<board>/jobs/123` y para las páginas propias con `?gh_jid=123`). Respaldo si no se puede extraer: buscar en `scraper_posts` la fila con `url = source_url` y usar su `post_id`.
- **Solo si no hay board+id** se hace HEAD (y GET si HEAD no se soporta) a `source_url`: 404/410 → roto; dominio caído (error de DNS/conexión) → roto *después de 2 revisiones*; redirección que termina en la página general de la empresa (ruta sin `/jobs/<id>`, o `?error=true`) → roto; 403/429/5xx/tiempo agotado → **desconocido**.
- VERIFICAR antes de confiar: probar la API con 3 vacantes que sepamos cerradas y 3 abiertas y confirmar el 404. Es lo primero que se hace.

PASOS

1. **Migración** `backend/sql/link-check-migration.sql`:

```
ALTER TABLE community_posts ADD COLUMN IF NOT EXISTS link_status TEXT
  CHECK (link_status IN ('ok', 'roto', 'desconocido'));
ALTER TABLE community_posts ADD COLUMN IF NOT EXISTS link_checked_at TIMESTAMPTZ;
ALTER TABLE community_posts ADD COLUMN IF NOT EXISTS link_status_reason TEXT;
-- Para la regla de «2 revisiones seguidas con 24 h o más de diferencia»:
ALTER TABLE community_posts ADD COLUMN IF NOT EXISTS link_fail_count INT NOT NULL DEFAULT 0;
ALTER TABLE community_posts ADD COLUMN IF NOT EXISTS link_first_failed_at TIMESTAMPTZ;
```

```
CREATE INDEX IF NOT EXISTS community_posts_link_check_idx
  ON community_posts (link_checked_at NULLS FIRST) WHERE type = 'job';
```

Como la tabla es la misma que usa todo el feed, el código del feed **no cambia** (no se lee `link_status` en ninguna consulta del feed).

2. **Módulo puro** `backend/services/vacancies/link-check.ts` con: `parseGreenhouseJobId(url)`, `classifyResponse({status, redirectedTo, networkError, timedOut}) → {status, reason}` y `nextState(previous, result, now)` (aplica la regla de 2 revisiones: una falla marca `link_fail_count = 1` y `link_first_failed_at` pero el estado visible sigue siendo el anterior; la segunda falla, con ≥ 24 h de diferencia, lo pasa a `roto`; un éxito reinicia el contador). Todo lo que decide va aquí, sin red, para poder probarlo.
3. **Ejecutor** `runLinkCheck({ limit })` en el mismo archivo: lee las abiertas (45 días, `type='job'`) **ordenadas por** `link_checked_at` **con las nulas primero**, en lotes de 50 con `readAll` / `.range`; por vacante usa `fetch` con `AbortController` (10 s), cabecera `User-Agent: AvoTalentLinkCheck/1.0 (+https://avotalent.app)`, `redirect: 'manual'` con tope de 2 redirecciones; espera 200 ms entre peticiones (≈5 por segundo); guarda el resultado con `update` por lote. Nunca lanza: un error de una vacante se anota como `desconocido` y sigue.
4. **Cron** en `backend/services/cron.service.ts`, copiando el patrón de `FEED_HIDE_UNSERVED`: `cron.schedule('0 10 * * *', …)` (04:00 Monterrey) **solo si** `process.env.LINK_CHECK === 'on'`. Cada corrida revisa hasta 400 vacantes (las más atrasadas); con ≈900 abiertas, en 3 días todas tienen estado y la regla «ninguna más de 7 días» se cumple de sobra. Además, las que quedaron con `link_fail_count = 1` se revisan en la corrida siguiente (≥ 24 h después) con prioridad.
5. **Ruta del panel** en `analytics.routes.ts`: `GET /links` devuelve `{ ready, checkedSince, open, byStatus: {ok, roto, desconocido, sinRevisar}, oldestCheckAgeDays, broken: [{ company, title, reason, checkedAt, slug }] }`. La **lista** de rotas lleva empresa y puesto (datos públicos de la vacante, **no personales**); para el agente, agregar `'/links'` a `SERVICE_READABLE` y la entrada `vacancy_links` en `PANEL_METRICS`. Sin la migración, responder `{ ready: false }`.
6. **Pantalla**: una tarjeta «Enlaces de postulación» en `apps/avocado/.../panel/oferta/page.tsx` (junto a «Feed visible»): conteos por estado y una `DataTable` con las rotas (empresa, puesto, motivo, fecha de revisión).
7. **Prueba local sin escribir**: `npm run link-check` (script nuevo `backend/scripts/link-check.ts`, por defecto en modo prueba: revisa 20 y solo imprime; `-- --apply` escribe).
8. **Anotar** en `CLAUDE.md` la viñeta y, al prender `LINK_CHECK=on` en Railway, `log-change` tipo `config`.

PRUEBAS OBLIGATORIAS (VITEST, `BACKEND/TESTS/UNIT/LINK-CHECK.TEST.TS`)

- Caso del criterio de cierre: **5 enlaces buenos, 3 rotos y 2 bloqueados** con `fetch` simulado → 5 `ok`, 3 `roto` (tras la segunda revisión), 2 `desconocido`.
- `parseGreenhouseJobId` con las dos formas de enlace (`/jobs/123` y `?gh_jid=123`) y con uno sin id (→ `null`).
- Una falla aislada no cambia el estado visible; dos con ≥ 24 h sí; un éxito en medio reinicia el contador.
- Tiempo agotado → `desconocido`, nunca `roto`. Redirección a la página general → `roto`.
- El ejecutor respeta el límite de ritmo y no lanza si una vacante falla.

REGLAS Y CIERRE

- No toca el trabajo de ocultado de las 9:30 (`refreshFeedVisibility`); no se importa en él. Respeta `robots.txt` en el respaldo HEAD/GET de páginas propias; la API de Greenhouse es pública y documentada. Sin proxies.
- **Cierre:** cada abierta con estado y fecha; ninguna sin revisar > 7 días; el panel muestra conteos y la lista; el agente lee `vacancy_links`; anotado en `get_change_log`. **Primera lectura:** % de abiertas rotas tras el lote del 20 nov.

Tarea 22 — Postulaciones por persona, no por clic

Lun 16 – mar 17 nov · esfuerzo S–M · es la base del 49 de la semana 2

RESPUESTAS A LAS PREGUNTAS ABIERTAS

- **¿Dónde se guarda la respuesta del punto 46?** HECHO En la tabla `vacancy_applications` (migración `vacancy-applications-migration.sql`). Columnas que importan: `user_id`, `vacancy_id`, `vacancy_slug`, `apply_host` (el sitio al que salió), `status` (`clicked | not_yet | applied | interview | offer | rejected`), `clicked_at` (cuándo tocó postular), `answered_at`, `applied_at`. **No** son eventos de analítica web: es una tabla de la aplicación.
- **¿Existe `user_id` en el «tocó postular»?** HECHO Sí: `vacancy_applications` nace en el momento en que una persona **con cuenta** toca postular (`POST /api/community/applications`), así que siempre trae `user_id`. El evento de analítica `click_start_apply` sí es de sesión, y por eso este cálculo **no debe salir de eventos**.
- **¿Se puede responder más de una vez por vacante?** HECHO Hay **una fila por (persona, vacante)** y cada respuesta la **sobrescribe** (`answerPatch` / `statusPatch` en `services/applications/applications.ts`). Por construcción, «cuenta la última respuesta» ya es lo que pasa; no hay que programar esa regla. La persona también puede cambiar el estado desde «Mis postulaciones» (entrevista, oferta, no avanzó): esos estados **suponen** que se postuló.
- **¿Cuántas respuestas hay ya?** HECHO (lectura del 13 nov) 63 filas de 46 personas distintas: **38** sin respuesta (`clicked`), **11** «sí me postulé» (`applied`, 7 personas distintas) y **14** «todavía no» (`not_yet`, 10 personas distintas). Primer toque: 3 nov. Faltan ≈5 respuestas para el umbral de «primera lectura útil» (≥ 30).

CAMBIOS RESPECTO AL DISEÑO

- **No se modifica `web_apply_funnel`** (su contrato lo leen el panel y el agente). Se agrega una **métrica aparte**: función SQL `analytics_apply_people`, ruta `GET /api/admin/analytics/web/apply-people`, métrica del agente `web_apply_people`. La vista actual por clics se conserva para comparar.
- Solo cuentas reales: `users.user_kind = 'user'`, `is_test_account = false`, `is_scraper_profile` no verdadero.

PASOS

1. **Migración** `backend/sql/apply-people-migration.sql`:

```
CREATE OR REPLACE FUNCTION analytics_apply_people(p_from timestamptz, p_to timestamptz)
RETURNS TABLE (tapped int, said_applied int, said_not_yet int, applied_pairs int, answered_pairs int,
               active_people int, first_answer_at timestamptz)
LANGUAGE sql STABLE AS $$
  WITH real AS (
    SELECT id FROM users
    WHERE user_kind = 'user' AND is_test_account = false AND is_scraper_profile IS NOT TRUE
  ),
  v AS (SELECT va.* FROM vacancy_applications va JOIN real r ON r.id = va.user_id)
  SELECT
    (SELECT COUNT(DISTINCT user_id) FROM v WHERE clicked_at >= p_from AND clicked_at < p_to)::int,
    (SELECT COUNT(DISTINCT user_id) FROM v WHERE status IN ('applied','interview','offer','rejected')
        AND applied_at >= p_from AND applied_at < p_to)::int,
    (SELECT COUNT(DISTINCT user_id) FROM v WHERE status = 'not_yet'
        AND answered_at >= p_from AND answered_at < p_to)::int,
    (SELECT COUNT(*) FROM v WHERE status IN ('applied','interview','offer','rejected')
        AND applied_at >= p_from AND applied_at < p_to)::int,
    (SELECT COUNT(*) FROM v WHERE status IN ('applied','interview','offer','rejected','not_yet')
        AND answered_at >= p_from AND answered_at < p_to)::int,
    (SELECT COUNT(DISTINCT d.user_id) FROM user_activity_daily d JOIN real r ON r.id = d.user_id
        WHERE d.day >= (p_from AT TIME ZONE 'America/Monterrey')::date
          AND d.day <= (p_to   AT TIME ZONE 'America/Monterrey')::date)::int,
    (SELECT MIN(answered_at) FROM v WHERE status IN ('applied','interview','offer','rejected','not_yet'));
```

```
$$;
GRANT EXECUTE ON FUNCTION analytics_apply_people(timestamptz, timestamptz) TO service_role;
```

Definiciones (documentarlas en la pantalla): **tocaron** = personas distintas con `clicked_at` en el periodo; **dijeron «sí me postulé»** = personas distintas con estado de postulada y `applied_at` en el periodo; **dijeron «todavía no»** = personas distintas con ese estado y `answered_at` en el periodo. Una persona con dos vacantes distintas puede estar en «sí» y en «todavía no» a la vez (son vacantes distintas); por eso se entregan también los **pares persona-vacante** (`applied_pairs`, `answered_pairs`).

2. **Servicio** `backend/services/analytics/apply-people.ts` (copiar la forma de `apply-funnel.ts`: `buildApplyPeople(row, days)` puro + `loadApplyPeople(days)` que llama `supabase.rpc` y devuelve `{ ready: false }` si falta la migración con `isMissingMigration`). Calcula: `responseRate = answered / tapped` (**siempre se muestra junto a las cifras**: es el sesgo que advierte el PDF), `appliedPerActivePerson = applied_pairs / active_people` y `answersSince = first_answer_at` (la pantalla dice «hay respuestas desde …»).
3. **Ruta** `router.get('/web/apply-people', handle('postulaciones por persona', …))` con `periodOf(req)` (7 o 30 días). Agregar `'/web/apply-people'` a `SERVICE_READABLE` y `web_apply_people` a `PANEL_METRICS`. **Solo conteos, nunca identificadores.**
4. **Por vacante o fuente** (criterio de cierre «si ya existe el dato»): sí existe — `vacancy_slug` y `apply_host`. Agregar una segunda función o un parámetro `group_by` con lista cerrada (`host`) que devuelva los mismos conteos por `apply_host`. Con menos de ≈50 respuestas no se concluye por fuente: la pantalla lo dice.
5. **Pantalla**: dentro de Tráfico web › Postulación (`panel/trafico/postulacion/page.tsx`) una tarjeta «Postulaciones por persona» con las tres cifras por 7 y 30 días, la tasa de respuesta y «hay respuestas desde <fecha>».

PRUEBAS OBLIGATORIAS

- SQL contra Postgres real (patrón PGlite de `tests/unit/provider-funnel.test.ts`): **3 casos** del criterio de cierre — (1) una persona toca 3 veces la misma vacante → cuenta 1; (2) responde «sí» y luego «todavía no» en la misma vacante → cuenta la última; (3) responde «sí» en una vacante y «todavía no» en otra → está en ambas cifras de personas y en los pares correspondientes. Más: cuentas de prueba y del scraper no cuentan; sin respuestas → ceros y tasa `null`, nunca división entre cero.
- `buildApplyPeople` puro: tasa de respuesta y postulaciones por persona activa.
- `analytics-access.test.ts`: `/web/apply-people` es legible por el agente y la respuesta no trae `user_id`.

Cierre: tres cifras por 7 y 30 días; casos de prueba verificados; `get_change_log`. **Primera lectura útil:** con ≥ 30 respuestas (hoy 25).

Tarea 56 — LinkedIn inteligente: agente en modo «proponer»

Mié 18 (tarde) – jue 19 nov · esfuerzo L · depende de la ruta de imagen del 32 · el post 4 (18:30) no se toca

RESPUESTAS A LAS PREGUNTAS ABIERTAS

- **¿Qué cuenta publica hoy?** HECHO El **perfil personal del fundador**, no la página de empresa: `publishVacancyPost` (`backend/services/linkedin/vacancy-publisher.ts`) publica con `urn:li:person:<linkedin_id>` y el token OAuth guardado en la tabla `linkedin_profiles` (la conexión más reciente). Publicar desde la página de empresa requiere otro permiso de LinkedIn: **fuera de alcance**.
- **¿Dónde aprueba el fundador?** En el **panel Telemetry** (sesión de superadmin). No por correo ni app: el correo gasta cupo de Resend y no hay app móvil.
- **¿Cómo se publica hoy y se puede programar el horario?** HECHO Con la API `ugcPosts` v2 de LinkedIn (`services/linkedin.service.ts`), como **enlace con tarjeta** (`ARTICLE`). La hora la programa **nuestro** cron, no LinkedIn: las horas viven solo en `backend/services/analytics/schedule.ts` (`PUBLICATION_SLOTS`: 8:30, 11:30, 14:30 y 18:30 Monterrey; cron en UTC = 14:30, 17:30, 20:30 y 00:30). Se enciende con `LINKEDIN_VACANCY_POSTS=on`.
- **¿Hay un código de vacante en el enlace y en qué parámetro?** HECHO Sí: `utm_content=<slug de la vacante>`, con `utm_source=linkedin&utm_medium=social&utm_campaign=vacante-auto`. Lo arma `vacancyUrl()` en `services/linkedin/vacancy-post.ts`. La medición de personas y registros por publicación se basa en ese `utm_content` (`loadSegments('linkedin_post', …)`). **Regla de seguridad:** el modelo **nunca escribe el enlace**; el sistema lo agrega con `vacancyUrl()`.
- **Tope de gasto diario / qué pasa si no aprueba:** ver decisiones D1 y D2 (sección 2).
- **«Clics de todos los posts»:** HECHO LinkedIn solo entrega clics por publicación de los 3 más recientes (Excel «Análisis conjunto»; no hay API para el perfil personal). Lo que **sí existe para todos los posts** es la medición propia: personas y registros por `utm_content`, que ya muestra el reporte de LinkedIn (`buildLinkedInReport` en `services/analytics/linkedin-report.ts`, con `inExcel: false` cuando LinkedIn no dio detalle). Entonces «ampliar» = mostrar esa lista completa, **por post, también para los de propuesta del agente**, y usarla como contexto del día siguiente. No hay que pedirle nada nuevo a LinkedIn.

ARQUITECTURA (RESUMEN)

```
06:30 Monterrey (12:30 UTC)  cron  -> propone 3 posts (slots 1,2,3)  -> tabla linkedin_proposals (estado
'propuesta')
                                       |  si la API falla / tope de gasto / validación falla -> respaldo con
la lógica actual (pickVacancy + buildPostText)
07:15 Monterrey (13:15 UTC)  cron  -> reintento idempotente (si ya hay 3 propuestas de hoy, no hace nada)
fundador, en el panel          ->  Aprobar | Editar y aprobar | Rechazar (motivo corto)
08:30 / 11:30 / 14:30          cron publicador existente (publishVacancyPost)
                                  slot 1-3 y modo 'propose' -> publica SOLO si hay propuesta
aprobada/editada; si no -> no publica y la marca 'expirada'
18:30 (slot 4)                 igual que hoy (experimento de formato gancho_slot4). El agente no lo toca.
```

PASOS

1. **Dependencia y variables.** En `backend/`: `npm i @anthropic-ai/sdk`. Variables en Railway (las pone el fundador; **nunca** escribirlas en un archivo ni en el chat): `ANTHROPIC_API_KEY`; `LINKEDIN_AGENT_MODE` = `off` (por defecto: se comporta exactamente como hoy) | `propose`; el valor `publish` **no se implementa esta semana**: si llega, se trata como `propose` y se escribe un aviso en el log. `LINKEDIN_AGENT_MODEL` (por defecto el modelo tipo Sonnet más reciente; verificar el identificador vigente en la documentación de Anthropic VERIFICAR), `LINKEDIN_AGENT_DAILY_BUDGET_USD` (por defecto `1.00`) y las dos constantes de precio por millón

de tokens (`ANTHROPIC_PRICE_IN_PER_MTOK`, `ANTHROPIC_PRICE_OUT_PER_MTOK`) para calcular el costo con `response.usage`.

2. **Migración** `backend/sql/linkedin-agent-migration.sql` (tres tablas nuevas, todas con RLS):

```
CREATE TABLE IF NOT EXISTS linkedin_agent_rules (
  version INT PRIMARY KEY, rules TEXT NOT NULL, note TEXT, created_at TIMESTAMPTZ NOT NULL DEFAULT now());

CREATE TABLE IF NOT EXISTS linkedin_proposals (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  for_date DATE NOT NULL,                -- día en Monterrey
  slot SMALLINT NOT NULL CHECK (slot BETWEEN 1 AND 3),
  vacancy_id UUID REFERENCES community_posts(id) ON DELETE SET NULL,
  vacancy_slug TEXT, company TEXT, role_category TEXT,
  text TEXT NOT NULL,                    -- texto propuesto, SIN enlace ni hashtags
  edited_text TEXT,                      -- el del fundador, si lo editó
  reason TEXT NOT NULL,
  image_url TEXT,                        -- la tarjeta OG que verá LinkedIn (tarea 32)
  status TEXT NOT NULL DEFAULT 'propuesta'
    CHECK (status IN ('propuesta','aprobada','editada','rechazada','publicada','expirada','fallida')),
  reject_reason TEXT,
  source TEXT NOT NULL DEFAULT 'agente' CHECK (source IN ('agente','respaldo')),
  rules_version INT, model TEXT, input_tokens INT, output_tokens INT, cost_usd NUMERIC(8,4),
  decided_at TIMESTAMPTZ, published_at TIMESTAMPTZ, linkedin_post_id TEXT,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  UNIQUE (for_date, slot));

CREATE TABLE IF NOT EXISTS linkedin_agent_calls (   -- cada llamada a la API: sirve para el tope de gasto
y para auditar
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(), called_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  for_date DATE NOT NULL, ok BOOLEAN NOT NULL, model TEXT, input_tokens INT, output_tokens INT,
  cost_usd NUMERIC(8,4), error TEXT);

ALTER TABLE linkedin_agent_rules ENABLE ROW LEVEL SECURITY;
ALTER TABLE linkedin_proposals ENABLE ROW LEVEL SECURITY;
ALTER TABLE linkedin_agent_calls ENABLE ROW LEVEL SECURITY;
REVOKE ALL ON linkedin_agent_rules, linkedin_proposals, linkedin_agent_calls FROM anon, authenticated;
INSERT INTO linkedin_agent_rules (version, rules, note) VALUES (1, '(las reglas de la sección «Reglas
guardadas» de esta guía)', 'versión inicial') ON CONFLICT DO NOTHING;
```

VERIFICAR que `linkedin_vacancy_posts.format` y `.experiment` no tengan `CHECK` que rechace valores nuevos (`linkedin-experiments-migration.sql`); las publicaciones del agente se anotan con `format = 'agente'` y `experiment = 'agente_linkedin'` para **no mezclarse con el experimento del post 4** (`loadExperiment` filtra por `experiment = 'gancho_slot4'`).

3. **Módulos nuevos** en `backend/services/linkedin/agent/` (todo lo que decide es **puro** y se prueba sin red):
   - `candidates.ts` — arma las candidatas **en código**, no el modelo: reutiliza `loadPool()` y `publishScore()` de `vacancy-post.ts`; filtra: vacantes **visibles** del feed (`hidden_from_feed` no verdadero), roles de `ROLE_FOCUS` con candidatas, **sin** vacantes ya en `linkedin_vacancy_posts`, empresa fuera de su *cooldown*, **no el mismo puesto/empresa que ayer** («sin repetir el mismo perfil dos días seguidos»), y —cuando exista la tarea 21— `link_status` distinto de `roto`. Entrega las mejores 30–40 ordenadas por `publishScore`.
   - `context.ts` — el **único** dato que sale hacia el modelo: por candidata `{ id, title, company, role, seniority, location, modality, skills (etiquetas), budget, ageDays }` (datos públicos de la vacante) + resultados **agregados** de los posts de los últimos 7 y 14 días `{ role, company, format, arrivals, registrations }` + el texto de las reglas vigentes y su versión. **Prohibido** incluir: ids de usuario, correos, nombres de personas, cifras de la comunidad, contenido de publicaciones de miembros. Una prueba verifica que el JSON enviado no contiene `@` ni claves `user`/`email`.
   - `schema.ts` — el formato de salida (abajo) y el **validador** `validateProposal(item, candidate)`.
   - `propose.ts` — `proposeDay(now)`: revisa el tope de gasto, llama a la API, valida, aplica el respaldo, **compone el texto final** y escribe en `linkedin_proposals` y `linkedin_agent_calls`.

- `approval.ts` — `approve | editAndApprove | reject` y `approvedProposalFor(date, slot)`.

4. **Llamada a la API (referencia)**. Se fuerza una salida estructurada con una herramienta y se valida con Zod (ya está en el proyecto):

```
const client = new Anthropic({ apiKey: process.env.ANTHROPIC_API_KEY })
const response = await client.messages.create({
  model: process.env.LINKEDIN_AGENT_MODEL ?? DEFAULT_MODEL,
  max_tokens: 2000,
  system: SYSTEM_PROMPT + '\n\nREGLAS (versión ' + rules.version + '):\n' + rules.rules,
  tools: [{ name: 'proponer_posts', description: 'Devuelve 3 propuestas de publicación de LinkedIn.',
    input_schema: { type: 'object', required: ['posts'], properties: { posts: { type: 'array', minItems:
3, maxItems: 3,
      items: { type: 'object', required: ['slot', 'vacancy_id', 'text', 'reason'], properties: {
        slot: { type: 'integer', enum: [1, 2, 3] }, vacancy_id: { type: 'string' },
        text: { type: 'string', maxLength: 900 }, reason: { type: 'string', maxLength: 300 } } } } } } }],
  tool_choice: { type: 'tool', name: 'proponer_posts' },
  messages: [{ role: 'user', content: JSON.stringify(context) }],
})
// salida: response.content.find(b => b.type === 'tool_use').input  ->  Zod  ->  validateProposal por cada
post
// costo: response.usage.input_tokens / output_tokens  x  precios configurables
// tiempo máximo de la llamada: 60 s (AbortSignal); un solo reintento si falla la validación (se agrega el
error al mensaje).
```

5. **Prompt del sistema (referencia, adaptable).**

```
Eres el editor de LinkedIn de AvoTalent, una comunidad en español de personas de tecnología que buscan
trabajo.
Recibes una lista de vacantes candidatas (datos públicos) y los resultados recientes de publicaciones
anteriores.
Elige 3 vacantes DISTINTAS (3 empresas distintas) para los turnos 1, 2 y 3 y escribe el texto de cada
publicación.
Para cada una devuelve: vacancy_id (solo de la lista), text y reason (por qué esa vacante, en una frase).
El text: español neutro de México, tono directo y cálido, 350 a 700 caracteres, empieza con un gancho
sobre el puesto o la empresa,
menciona puesto, empresa y lo más útil para decidir (ubicación/modalidad, sueldo SOLO si viene en los
datos), y cierra invitando a ver la vacante.
NO incluyas enlaces, hashtags ni emojis de más (máximo 3). NO uses números que no estén en los datos de la
vacante.
NO prometas nada que la plataforma no cumple: no digas "100 vacantes", "miles de vacantes", ni menciones
filtros, alertas, mensajes a reclutadores
ni cifras de la comunidad. NO cites a personas. Si los resultados recientes muestran qué tipo de puesto o
formato trajo más visitas, úsalo como
criterio de elección, sin inventar causas: es correlación con pocos datos.
```

6. **Validación (obligatoria, en código).** Un post se acepta solo si cumple **todo**: `vacancy_id` está en la lista enviada; las 3 vacantes y las 3 empresas son distintas; `text` mide entre 200 y 900 caracteres; **no contiene** `http`, `www.`, `@` ni `#`; no coincide con la lista prohibida (`/\b100\s+vacantes\b/i`, `/\bmiles\s+de\b/i`, `/\bfiltros?\b/i`, `/\bcomunidad\s+de\s+\d/i`, `/garantiz/i`, `/\bsin\s+experiencia\b/i` y las que vaya sumando la regla guardada); **todo número** del texto aparece en los datos de esa vacante (título, sueldo…); a lo sumo 3 emojis; `reason` ≤ 300. Si un post falla: un reintento con el motivo; si vuelve a fallar, **ese turno usa el respaldo**.
7. **Texto final = lo que sale a LinkedIn.** El sistema compone: `texto del agente` + línea en blanco + 👉 `${vacancyUrl(base, slug)}` + línea en blanco + `hashtagsFor(vacancy)` (mismas funciones que usa `buildPostText`, así el enlace y el `utm_content` son idénticos a hoy). El **fundador ve y edita el texto del agente**; el enlace y los hashtags se ven en la vista previa pero no se editan.
8. **Respaldo.** Si no hay `ANTHROPIC_API_KEY`, la API falla, se agota el tiempo (60 s), se pasa del tope de gasto del día o la validación falla dos veces: se genera la propuesta con la lógica actual (`pickVacancy` + `buildPostText`) con `source = 'respaldo'`. **También requiere aprobación.** Nunca se publica algo sin pasar por «Aprobar».

9. **Tope de gasto.** Antes de llamar: `SUM(cost_usd)` de `linkedin_agent_calls` del día (Monterrey) + una estimación conservadora de esta llamada > `LINKEDIN_AGENT_DAILY_BUDGET_USD` → no se llama y se usa el respaldo. Toda llamada, buena o mala, se anota.
10. **Gancho en el publicador existente.** En `publishVacancyPost(options)`, justo después de los guardas actuales (intervalo de 90 minutos y `COMMUNITY_APP_URL`) y solo si `!options.slug && !options.dryRun`:

```
const mode = process.env.LINKEDIN_AGENT_MODE === 'propose' || process.env.LINKEDIN_AGENT_MODE ===
'publish' ? 'propose' : 'off'
const slot = slotOf(new Date())            // services/linkedin/experiment.ts
if (mode === 'propose' && slot >= 1 && slot <= 3) {
  const proposal = await approvedProposalFor(todayInMonterrey(), slot)     // estado 'aprobada' o
'editada'
  if (!proposal) { await expirePending(todayInMonterrey(), slot); return { status: 'skipped', reason:
'modo proponer: sin propuesta aprobada para este turno' } }
  // seguir con el flujo normal usando proposal.vacancy / el texto final; al terminar -> estado
'publicada' + linkedin_post_id
}
// slot 4 y modo 'off': exactamente el comportamiento actual
```

    La fila de `linkedin_vacancy_posts` se sigue creando **antes** de publicar (reserva la vacante por el `UNIQUE(vacancy_id)`), con `post_text` = texto final y las columnas `format = 'agente'`, `experiment = 'agente_linkedin'`, `slot`. Si LinkedIn falla, la propuesta queda `fallida` y el error se ve en el panel.
11. **Cron de la propuesta** en `cron.service.ts`: `cron.schedule('30 12 * * *', …)` y `('15 13 * * *', …)` (reintento), ambos con `LINKEDIN_AGENT_MODE === 'propose'` y capturando errores. `proposeDay` es **idempotente**: si ya hay propuestas para (`for_date`, `slot`) no las duplica (el `UNIQUE` lo garantiza).
12. **Rutas de aprobación** (solo superadmin, bajo `/api/admin/analytics`; **no** en `SERVICE_READABLE`): `GET /linkedin-proposals?date=` (las del día con la vista previa de la vacante, la imagen y el estado), `POST /linkedin-proposals/:id/approve`, `POST /linkedin-proposals/:id/edit` (`{ text }`, pasa por el mismo validador y queda `editada`), `POST /linkedin-proposals/:id/reject` (`{ reason }`, ≤ 120 caracteres). Solo se puede decidir mientras el turno no haya pasado; después, `expirada`. Validar que `:id` sea un UUID.
13. **Pantalla** `apps/avocado/src/app/(panel)/panel/linkedin-propuestas/page.tsx` (+ entrada en `NAV_GROUPS`): arriba el modo actual (`propose` / `off`) y el gasto del día contra el tope; tres tarjetas (hora del turno, empresa · puesto, **imagen de la tarjeta** con `<img src={image_url}>`, texto editable, razón, botones **Aprobar**, **Editar y aprobar**, **Rechazar** con campo de motivo corto, y una etiqueta «propuesta del agente» o «respaldo»); debajo, el historial de los últimos días con el resultado de cada post (personas y registros por `utm_content`) y el indicador **«aprobadas sin cambio / total»**.
14. **Alarmas del panel** (`services/analytics/alerts.ts`): hoy `posts-missed` compara `expectedPublicationsToday` contra lo publicado; en modo proponer los turnos 1–3 sin aprobar **no son un fallo**. Ajustar `expectedToday` para que cuente el turno 4 más los turnos 1–3 **aprobados**, y agregar dos avisos: «Hay propuestas por aprobar» (faltan < 60 minutos para el turno) y «Un turno no se publicó por falta de aprobación».
15. **Reglas guardadas** (tabla `linkedin_agent_rules`, versión 1, editable solo por migración/script). Texto inicial: (1) 4 posts al día como máximo; los turnos 1–3 son del agente, el 4 es del experimento. (2) Solo vacantes visibles del feed, de puestos con candidatas. (3) No repetir empresa dentro del *cooldown* ni el mismo puesto/empresa dos días seguidos. (4) Sin números, promesas ni cifras de comunidad sin respaldo en los datos. (5) Sin enlaces ni hashtags en el texto. (6) La elección usa los resultados de 7 y 14 días como contexto, nunca como causa. **Sin entrenamiento de modelos:** mejora por contexto y reglas guardadas.
16. **Resultado por post** (criterio «el panel muestra el resultado de todos los publicados»): `GET /linkedin-proposals/results` o ampliar `buildLinkedInReport` para listar **todas** las publicaciones medidas (no solo las 3 con detalle de LinkedIn), con personas, registros y fuente (`agente` / `respaldo` / `plantilla`). Es la misma fuente que alimenta el contexto del día siguiente.

PRUEBAS OBLIGATORIAS

- **Validador**: rechaza enlace, `#`, `@`, «100 vacantes», un número que no está en los datos, texto de < 200 o > 900 caracteres, `vacancy_id` inventado, empresas repetidas.
- **Candidatas**: excluye ocultas, ya publicadas, rol fuera de foco, la misma empresa/puesto de ayer y (si existe) enlace roto.
- **Contexto**: el JSON enviado no contiene correos, nombres ni ids de usuario (prueba de inspección).
- `proposeDay` **con la API simulada**: buena → 3 propuestas; salida inválida → un reintento y luego respaldo; API caída/tiempo agotado → respaldo; tope de gasto superado → ni llama; idempotente (segunda corrida no duplica).
- **Aprobación**: solo se aprueba/edita/rechaza lo propuesto y antes de la hora; editar pasa por el validador; nadie sin sesión de superadmin.
- **Publicador**: modo `off` = comportamiento idéntico al actual (las pruebas existentes siguen verdes); modo `propose` y turno 1–3 sin aprobar → `skipped` y la propuesta pasa a `expirada`; aprobada → se publica con el enlace `vacancyUrl()` (con `utm_content`); turno 4 no cambia.
- Prueba de despliegue en seco: con `LINKEDIN_AGENT_MODE=propose` y **sin** `LINKEDIN_VACANCY_POSTS=on` (publicador apagado) se generan y aprueban propuestas sin publicar nada: así se prueba todo el circuito **sin riesgo** (no hay staging).

DESPLIEGUE POR FASES (PORQUE NO HAY STAGING)

1. Subir con `LINKEDIN_AGENT_MODE` sin definir (= `off`): nada cambia. El fundador corre el SQL.
2. Día 1: `LINKEDIN_AGENT_MODE=propose` con `LINKEDIN_VACANCY_POSTS` **apagado**: se revisan las propuestas en el panel durante un día, sin publicar.
3. Día 2: se prende `LINKEDIN_VACANCY_POSTS=on`: los turnos 1–3 publican **solo lo aprobado**; el 4 sigue igual.
4. Anotar cada interruptor con `log-change` (tipo `config`, hora en UTC).

Fuera de alcance: modo publicar sin aprobación; carruseles; publicar desde la página de empresa; Google Imágenes; entrenar modelos. **Cierre:** cada mañana hay 3 propuestas con texto, razón e imagen antes de las 8:30; aprobar publica en su horario con el código de la vacante; lo no aprobado no se publica; el 4 sigue igual; hay tope de gasto y de posts; si el agente falla, respaldo; el panel muestra el resultado por post; anotado en `get_change_log`. **Medición:** los 7 y 14 días de LinkedIn contra la semana base, **como correlación y no causa** (el 32 cambia las vistas previas en las mismas semanas y no se podrán separar); «aprobadas sin cambio / total» desde la primera semana.

Tarea 32 — Vista previa al compartir (Open Graph y Twitter)

Mié 18 (la ruta de imagen) y jue 19 – vie 20 nov (el resto) · esfuerzo M · la plantilla de imagen la usa también el 56

RESPUESTAS A LAS PREGUNTAS ABIERTAS

- **¿La app renderiza en servidor o es de una sola pantalla en el navegador?** HECHO `apps/community` es **Next 16.3.3 con App Router**: las etiquetas salen en el HTML inicial **sí** se declaran en un componente de servidor. La página de la vacante es un componente de cliente, por eso sus metadatos **ya viven en un `layout.tsx` de servidor** (`app/(main)/vacantes/[slug]/layout.tsx`) y leen la versión pública `getSeoVacancy(slug)`. Estado actual por tipo de página:

Página	Hoy
Home (`app/(main)/page.tsx`)	Componente de servidor **sin metadatos propios**: hereda título y descripción de `app/layout.tsx`; sin `og:image`, sin `og:url`, sin Twitter.
Vacante	Tiene `openGraph` (título, descripción, `siteName`, tipo) pero la imagen es el **logo de la empresa** si existe; `twitter.card = 'summary'` (cuadrada, no grande); sin `locale`, `url` ni tamaño de imagen.
Listados (`/empleos`, `/empleos/[skill]`, `/puestos`, `/puestos/[role]`)	Tienen título/descripción/`openGraph`/`twitter: summary` pero **sin imagen**.
Empresa (`app/(main)/empresas/[company]/page.tsx`)	Componente de **cliente** (`'use client'`) sin `layout.tsx` ni `generateMetadata`: un rastreador no ve nada. **No es por el muro**: el endpoint `GET /api/community/companies/:slug` ya es público (a invitados les devuelve las vacantes reducidas). Falta solo el layout de servidor.

- **¿Qué campos son visibles a invitados?** HECHO Los de `SAFE_FIELDS` en `backend/src/routes/community/seo.routes.ts`, que son los que entrega `GET /api/community/seo/vacancy/:slug`: `id, slug, title, company, company_logo, location, modalidad, seniority_level, role_category, skills, budget, created_at` más `description` (HTML sin contacto) y `expired` (vencida a los 45 días). **El rango de sueldo (`budget`) ya es público**, así que se puede mostrar en la tarjeta. **Nada más** (ni enlace de postulación ni contactos).
- **¿Hay un renderizador de imágenes en el servidor?** HECHO No hay ninguno instalado, pero **no hace falta instalar nada**: `next/og` (`ImageResponse`, basado en Satori) viene dentro de Next. Va en una ruta de la propia app de comunidad (Vercel). Limitaciones de Satori que hay que respetar: solo **flexbox** (sin grid), cada `div` con más de un hijo lleva `display: 'flex'`, imágenes remotas con URL absoluta, tipografía por defecto salvo que se incluya un archivo .ttf (primera entrega: la por defecto).
- **¿De dónde salen los logos y con qué licencia?** HECHO `community_posts.company_logo` sale de `scraper_sources.metadata.storage_logo_url` (copias guardadas en nuestro almacenamiento; ver `sync.ts`). 685 de las 905 vacantes abiertas tienen logo. **La licencia no está verificada** y hoy ya se usan como imagen al compartir. Decisión D3: apagados por defecto (`OG_SHOW_LOGOS=off`) → insignia con la inicial.
- **¿Hay caché o CDN delante?** HECHO La comunidad corre en Vercel: tiene CDN. Una ruta que responde con `Cache-Control: public, s-maxage=…` se cachea en el borde. El backend (Railway) **no** tiene CDN; solo la caché en memoria `publicCache`. Además `getSeoVacancy` ya usa `next: { revalidate: 3600 }`, así que una vacante que vence puede tardar hasta 1 hora en verse «cerrada» en la tarjeta (aceptable).

PASOS (PRIMERO LO QUE NECESITA EL 56)

1. **Plantilla compartida** `apps/community/lib/og-card.tsx`: dos piezas. (a) **Pura** `cardProps(vacancy | null)` → `{ kind: 'vacante' | 'neutral', title, company, meta, badgeInitial, logoUrl }`: si no hay vacante, o `expired`, devuelve la tarjeta **neutral** (título de la vacante si se conoce, texto «Ver vacantes parecidas», **sin sueldo ni detalles**); si la vacante está abierta, título «*Puesto* en *Empresa*», meta (`modalidad · ubicación · sueldo` solo si son datos públicos y conocidos; descartar «No especificado»), inicial de la empresa, y `logoUrl` **solo si** `process.env.OG_SHOW_LOGOS === 'on'`. (b) El componente JSX de 1200×630: fondo azul marino (`#0f172a`), acento esmeralda (`#10b981`), marca «Avo**Talent**» (Avo blanco, Talent esmeralda, Inter 800 como el resto de la marca), título grande (máx. 2 líneas con recorte), empresa, meta y la insignia circular con la inicial a la derecha.
2. **Ruta de imagen** `apps/community/app/og/vacante/[slug]/image.png/route.tsx` (URL final `/og/vacante/<slug>/image.png`):

   ```
   import { ImageResponse } from 'next/og'
   export async function GET(_req: Request, { params }: { params: Promise<{ slug: string }> }) {
     const { slug } = await params
     const props = cardProps(await getSeoVacancy(slug))          // nunca revienta: sin vacante = neutral
     return new ImageResponse(<OgCard {...props} />, { width: 1200, height: 630,
       headers: { 'Cache-Control': 'public, max-age=3600, s-maxage=86400, stale-while-revalidate=604800' } })
   }
   ```

   Una ruta hermana `/og/default/image.png` con la tarjeta de marca (para home, listados y empresas) y, opcional, `/og/empresa/[slug]/image.png`. **Esta ruta (paso 2) es lo que el 56 necesita el miércoles.**
3. **Ayudante de metadatos** `apps/community/lib/og.ts`: `ogMetadata({ title, description, path, image })` que devuelve `title`, `description`, `alternates.canonical`, `openGraph` con `title, description, url, siteName: 'AvoTalent', locale: 'es_MX', type: 'website', images: [{ url, width: 1200, height: 630, alt }]` y `twitter: { card: 'summary_large_image', title, description, images: [url] }`. Una sola función para las 4 clases de página, para que no se desfasen. La descripción se recorta a ≈125 caracteres en límite de palabra (`ogDescription()`, pura).
4. **`metadataBase`** en `app/layout.tsx`: `metadataBase: new URL(SITE_URL)` (importar `SITE_URL` de `lib/seo.ts`). Sin esto Next no resuelve bien las URL relativas de imagen. Conservar el texto actual de la descripción de la home («La comunidad en español donde las conversaciones técnicas encuentran nuevas oportunidades laborales.»): cumple la decisión del 11 nov / punto 39 (sin prometer filtros ni «100 vacantes»).
5. **Aplicar a las 4 clases de página.** (a) **Home**: `export const metadata = ogMetadata({ … image: '/og/default/image.png' })` en `app/(main)/page.tsx`. (b) **Vacante**: reemplazar el bloque `openGraph`/`twitter` del `layout.tsx` por `ogMetadata` con `image = /og/vacante/<slug>/image.png` y **quitar** `images: [vacancy.company_logo]`; título «*Puesto* en *Empresa*»; vacante vencida o inexistente → metadatos neutrales (nombre de la vacante y «Ver vacantes parecidas», sin sueldo). Alinear también `cardFor()` en `backend/services/linkedin/vacancy-post.ts` (hoy «Puesto — Empresa») para que el título de la tarjeta de LinkedIn coincida. (c) **Listados**: agregar `ogMetadata` en los `generateMetadata` existentes de `empleos`, `empleos/[skill]`, `puestos` y `puestos/[role]` con la imagen por defecto. (d) **Empresa**: nuevo `app/(main)/empresas/[company]/layout.tsx` de servidor con `generateMetadata` que lee un endpoint público ligero nuevo `GET /api/community/seo/company/:slug` (`seo.routes.ts`; solo `{ name, bio recortada, openVacancies }`, **sin contactos**, con `REVALIDATE_SECONDS`); si no existe → neutral.
6. **El enlace que se comparte no cambia.** Los enlaces de los posts siguen siendo `/vacantes/<slug>?utm_source=linkedin&utm_medium=social&utm_campaign=vacante-auto&utm_content=<slug>`; `og:url`/canónico van **sin** parámetros (como ya hace el layout con `alternates.canonical`). El post pasa su URL explícita como `originalUrl` a LinkedIn, así que el clic conserva el `utm_content`. **VERIFICAR** en el Post Inspector de LinkedIn que el clic conserva los parámetros.
7. **Robots.** `app/robots.ts` permite `/` y solo bloquea rutas de cuenta: `/og/` queda accesible. No hay `middleware`/`proxy` que intercepte rastreadores.

PRUEBAS Y VERIFICACIÓN

- **Vitest (comunidad)** `tests/og-metadata.test.ts`: `ogMetadata` devuelve `locale: 'es_MX'`, imagen 1200×630 con URL absoluta, `twitter.card = 'summary_large_image'`; `ogDescription` ≤ 125 caracteres y sin cortar palabras; `cardProps`: abierta → título «Puesto en Empresa» con sueldo si es público; **vencida** → **neutral y sin sueldo**; inexistente → neutral; logo solo con `OG_SHOW_LOGOS=on`; «No especificado» no aparece. El `generateMetadata` de la vacante (con `getSeoVacancy` simulado) no incluye `company_logo` como imagen.
- **Fuente de la página** (criterio «revisado en la fuente»), con los rastreadores reales como User-Agent:

  ```
  curl -s -A "LinkedInBot/1.0" https://avotalent.app/vacantes/<slug> | grep -iE "og:|twitter:"
  curl -s -A "facebookexternalhit/1.1" https://avotalent.app/ | grep -iE "og:|twitter:"
  curl -sI https://avotalent.app/og/vacante/<slug>/image.png   # 200, image/png, cache-control presente
  ```

  Repetir con una vacante **vencida** (tarjeta neutral, sin sueldo) y con un `slug` inexistente.
- **Validadores:** Depurador de Meta (*Sharing Debugger*), *Post Inspector* de LinkedIn (pulsar «Inspect» para refrescar la caché de LinkedIn; ahí se ve la imagen) y una **prueba real en WhatsApp**. Para X: el validador público dejó de ser confiable **VERIFICAR**; se comprueba redactando un post con el enlace.
- La imagen no se prueba con Vitest (Satori necesita runtime de Next): se prueba abriendo la URL en el navegador con 3 casos (abierta con sueldo, abierta sin sueldo, vencida).

Reglas: una sola plantilla (compartida con el 56); sin datos que hoy estén detrás del muro; sin prometer filtros ni «100 vacantes». **Fuera de alcance:** JSON-LD y SEO/GEO (punto 28); imágenes con IA. **Cierre:** las 4 clases de página con etiquetas completas; verificadas en los validadores; vacante cerrada → tarjeta neutral; anotado en `get_change_log` (`feature`, `Expects: compartidos y clics de LinkedIn a 14 días`). **Riesgo:** mostrar un dato que no debía ser público → por eso la plantilla solo lee `getSeoVacancy`.

Tarea 45 — Perfil y Conjunto de perfiles en `people_value`

Vie 20 nov · esfuerzo S–M · si no hay capacidad, pasa a la semana 2 · solo se ve y se guarda: no cambia ningún correo, aviso ni invitación

RESPUESTAS A LAS PREGUNTAS ABIERTAS

- **Nombre exacto de cada estado.** HECHO Claves internas (en `EngagementState`, `services/analytics/engagement-history.ts`) y etiquetas (`STATE_LABEL`):

Clave	Etiqueta
`nueva`	Nueva
`activa`	Activa
`comprometida`	Comprometida
`en_riesgo`	En riesgo
`dormida`	Dormida
`reactivada`	Reactivada

  La tabla `engagement_snapshots` **no guarda el estado**: se calcula al leer con `engagementState()` a partir de la serie de fotos semanales. Esta tarea agrega las columnas para guardarlo.
- **¿Los días activos distintos ya se calculan por persona en 7 días?** HECHO Sí: `PersonValueRow.activeDays7` (y `activeDays7List`), días distintos de actividad en los últimos 7 días de Monterrey, de `user_activity_daily` (`buildPeopleValue` en `services/analytics/people-value.ts`). Ya aparece en la tabla como «Días activos (7 d)».
- **¿Conjunto como campo propio o calculado al leer? Las dos cosas, con una sola función.** Se **guarda** en la foto del lunes (el diseño lo exige: «se guardan», y es lo que permite ver el pasado) y se **calcula** en la vista viva con **la misma función pura** (`classifyProfile`), para que no puedan discrepar. Lo oficial para tratar a las personas es lo guardado el lunes; lo vivo es informativo (ver «Cómo se muestra»).
- **¿Qué cuenta como «participación» en «comprometida»?** HECHO En `engagementState`: una semana tiene participación si **discusiones + comentarios + votos > 0**, y «comprometida» exige participación en **2 o más semanas distintas** de las últimas 4 (más actividad en 3 de esas 4). **Hoy nadie participa** (0 discusiones, 0 comentarios y 0 votos de cuentas reales): por eso **nadie puede salir «comprometida»** y el conjunto 3 (Núcleo) solo tiene activas. Es lo esperado, no un error. No se cambia la regla en esta tarea.
- **Sobre «foto del 2 nov: 82 nuevas, 4 dormidas, 1 activa».** HECHO Coincide con la foto semanal del 2 nov (primera semanal, creada el lunes 9). **Hoy (13 nov, 108 cuentas reales):** 103 nuevas, 4 dormidas, 1 activa.

REGLAS (DECIDIDAS POR EL FUNDADOR; IMPLEMENTAR TAL CUAL)

Temperatura (sustituye la actual, que solo mira los puntos): **caliente** = enganche ≥ 6 **y** ≥ 2 días activos distintos; **tibia** = enganche de 2 a 5, **o** ≥ 6 con menos de 2 días activos (el caso que el PDF marcó como suposición: «queda tibia»); **fría** = enganche < 2. (El «puntos» del PDF es el **enganche**, que ya incluye +1 por día activo; ver tarea anterior de la semana pasada.)

Estado	Temperatura	Conjunto
nueva	caliente	**1** Prometedoras
nueva	tibia con ≥ 2 días activos	**1** Prometedoras
nueva	tibia con < 2 días activos	**2** No enganchadas
nueva	fría	**2** No enganchadas

activa o comprometida	caliente o tibia	**3** Núcleo
activa o comprometida	fría	**5** En riesgo o dormidas
reactivada	cualquiera	**4** Reactivadas
en riesgo	cualquiera	**5** En riesgo o dormidas
dormida	fría	**5** En riesgo o dormidas
dormida	tibia o caliente	«imposible» en el PDF — **existe**, ver caso D4 abajo

Caso D4 — «dormida con tibia o caliente» sí ocurre (1 cuenta hoy). El estado sale de las fotos semanales cerradas (la última es la del 2 nov: «dormida» = sin actividad en las 4 semanas anteriores) y la temperatura sale de lo que hizo en los **últimos 7 días**. Una persona que estaba dormida y volvió a hacer cosas esta semana queda dormida (por el estado, con retraso de hasta una semana) pero tibia (por lo reciente). **Recomendación:** asignarla al conjunto **4 (Reactivadas)** y guardar `state_lag = true` para distinguirla; no ocultarla ni lanzar un error. Es decisión del fundador (D4); mientras tanto implementar con esa recomendación.

PASOS

1. **Una sola función pura** en `backend/services/analytics/people-value.ts` (o `people-profile.ts` importado desde ahí). Constantes en el mismo lugar que `TEMPERATURE` (regla del PDF: «un solo lugar»):

```
export const TEMPERATURE = { hotAtLeast: 6, warmAtLeast: 2, hotMinActiveDays: 2 } as const

export function temperatureOf(engagement: number, activeDays: number): Temperature {
  if (engagement >= TEMPERATURE.hotAtLeast && activeDays >= TEMPERATURE.hotMinActiveDays) return
'caliente'
  if (engagement >= TEMPERATURE.warmAtLeast) return 'tibio'      // incluye 6+ con menos de 2 días activos
  return 'frio'
}
export type ProfileSet = 1 | 2 | 3 | 4 | 5
export const SET_LABEL: Record<ProfileSet, string> = { 1: 'Prometedoras', 2: 'No enganchadas', 3:
'Núcleo', 4: 'Reactivadas', 5: 'En riesgo o dormidas' }
export function classifyProfile(state: EngagementState, temp: Temperature, activeDays: number): { set:
ProfileSet; stateLag: boolean } {
  if (state === 'reactivada') return { set: 4, stateLag: false }
  if (state === 'nueva') return { set: temp === 'caliente' || (temp === 'tibio' && activeDays >= 2) ? 1 :
2, stateLag: false }
  if (state === 'activa' || state === 'comprometida') return { set: temp === 'frio' ? 5 : 3, stateLag:
false }
  if (state === 'en_riesgo') return { set: 5, stateLag: false }
  /* dormida */ return temp === 'frio' ? { set: 5, stateLag: false } : { set: 4, stateLag: true }   // D4
}
// Etiqueta del Perfil: «estado · temperatura» con la temperatura en femenino (la cuenta): 'nueva ·
tibia', 'dormida · fría'.
```

   Actualizar los **llamadores** de `temperatureOf` (`buildPeopleValue` pasa `activeDays7`) y el **texto de** `reasonOf`: para enganche ≥ 6 con < 2 días: «Tibio: sumó N puntos de uso en los últimos 7 días pero solo estuvo activa X día(s); caliente pide al menos 2 días activos distintos». Actualizar `tests/unit/people-value.test.ts` a la nueva firma.

2. **Migración** `backend/sql/profile-set-migration.sql`:

```
ALTER TABLE engagement_snapshots ADD COLUMN IF NOT EXISTS engagement_state TEXT;
ALTER TABLE engagement_snapshots ADD COLUMN IF NOT EXISTS profile TEXT;          -- p. ej. 'nueva · tibia'
ALTER TABLE engagement_snapshots ADD COLUMN IF NOT EXISTS profile_set SMALLINT CHECK (profile_set BETWEEN
1 AND 5);
ALTER TABLE engagement_snapshots ADD COLUMN IF NOT EXISTS state_lag BOOLEAN;
```

   Las fotos anteriores quedan con estas columnas en `NULL` (no se reescriben: «nunca sobrescribir»).

3. **Guardarlos en la foto del lunes.** En `takeWeeklySnapshot` (`engagement-history.ts`): después de insertar las filas de la semana, calcular el **estado** de cada persona con `loadEngagementStates(now)` (ya incluye la foto recién guardada), la temperatura (ya viene en `row.temperature`) y `classifyProfile`; y hacer un `update` por lote de **solo estas cuatro columnas** sobre las filas `kind = 'semanal'` de esa `week_start` y `profile IS NULL`. Tolerar que la migración aún no exista (si falla por columna, se omite con aviso en el log; la foto base de los demás campos no se pierde).
4. **La vista de `people_value`.** `GET /people-value` agrega por persona `profile`, `profileLabel`, `set`, `setLabel`, `stateLag`, usando `classifyProfile(state, row.temperature, row.activeDays7)` con el estado de `loadEngagementStates` (la ruta ya lo carga: `state`). Tabla de `apps/avocado/.../panel/personas/page.tsx`: dos columnas nuevas, **Perfil** (etiqueta de color de la temperatura + el estado) y **Conjunto** (1 a 5 con su nombre); filtro por conjunto además del de temperatura; el **CSV** suma `perfil`, `conjunto` (y el `estado` que ya trae).
5. **Cómo se muestra (para que nadie cambie de trato a diario).** Lo **oficial** es lo guardado en la última foto semanal que ya tenga esas columnas. Hasta que exista la primera (el lunes **23 nov** si se sube esta semana), la tabla muestra el cálculo vivo con la etiqueta *«provisional»*. En el detalle de la persona se ve siempre el vivo y, si existe, el del lunes.
6. **Métrica para el agente** (solo conteos): ampliar `GET /people-value/summary` (ya en `SERVICE_READABLE`, métrica `people_value`) con `byProfile` (conteo por `estado · temperatura`), `bySet` (conteo por conjunto 1 a 5, más `anomalies` para el caso D4) y `activeDaysBuckets` `{ '0': n, '1': n, '2': n, '3+': n }`. **El total de cada uno debe ser igual al de cuentas reales** (sin equipo, pruebas ni bots): hay una prueba que lo exige. Actualizar la descripción en `packages/mcp-analytics/src/panel.ts`.
7. **Comparación antes/después** (criterio de medición): un script `npm run profile-compare` (como el de `--baseline` del historial) que imprima la distribución con la regla vieja y la nueva sobre los mismos datos. Registrarlo en el `CLAUDE.md`.

PRUEBAS OBLIGATORIAS

- **Caso del criterio de cierre:** 6 puntos de enganche y 1 día activo → **tibia**; 6 puntos y 2 días → **caliente**. Más: 5 y 3 días → tibia; 1 y 1 → fría; 2 → tibia (límite); 0 → fría.
- **Tabla completa de `classifyProfile`** (las 10 filas de la tabla de arriba, cada una como un caso) y el caso D4 (`{ set: 4, stateLag: true }`).
- **Suma:** los conteos de `byProfile`, `bySet` y `activeDaysBuckets` suman el total de cuentas reales; las cuentas de prueba, del scraper y el equipo no entran.
- La foto del lunes guarda las 4 columnas y **no sobrescribe** una foto que ya existe (correr el trabajo dos veces no cambia nada). Sin la migración no lanza.
- `analytics-access.test.ts` sigue verde: la lista con correos no es legible con la llave del agente; el resumen sí.

VALORES DE REFERENCIA PARA COMPROBAR EL TRABAJO (DATOS REALES DEL 13 NOV, 108 CUENTAS)

Medida	Resultado esperado hoy
Estados	103 nuevas · 4 dormidas · 1 activa
Temperatura con la regla nueva	8 calientes · 39 tibias · 61 frías (con la regla actual serían 15 calientes: **7 cuentas bajan de caliente a tibia** por tener 1 solo día activo)
Conjuntos	1: 9 · 2: 94 · 3: 1 · 4: 0 · 5: 3 · caso D4 (dormida tibia): 1
Días activos en 7	0: 64 · 1: 35 · 2: 6 · 3 o más: 3

Pueden variar un poco porque entran cuentas nuevas cada día; lo que no debe variar es que **cada serie sume el total**. **Aviso para el panel y el agente:** bajar filas de caliente a tibia **parece una caída pero es un cambio de**

definición: anotarlo en `get_change_log` con una descripción que lo diga (como se hizo con la exclusión de bots), tipo `config`. **Fuera de alcance:** el trato por conjunto en correo, push o invitaciones (espera la respuesta del abogado); cambiar los puntos de las acciones. **Medición:** regreso por conjunto, primera lectura útil ≈ 7 dic.

Cierre — qué se corre, qué se prende y qué devuelve Francisco

A. Migraciones SQL que corre el fundador (en este orden, una por una)

Archivo (nuevo)	Tarea	Cuándo
`backend/sql/link-check-migration.sql`	21	Antes de prender `LINK_CHECK=on`
`backend/sql/apply-people-migration.sql`	22	Después de subir el código (hasta entonces la métrica responde «no está lista»)
`backend/sql/linkedin-agent-migration.sql`	56	Antes de poner `LINKEDIN_AGENT_MODE=propose`
`backend/sql/profile-set-migration.sql`	45	Antes del lunes 23 nov (primera foto con perfil y conjunto)

La tarea 32 **no necesita SQL**. Todas son idempotentes: se pueden correr dos veces sin daño.

B. Variables y interruptores (Railway / Vercel) — todos apagados por defecto

Variable	Dónde	Para qué
`LINK_CHECK=on`	Railway	Enciende la revisión diaria de enlaces (21).
`ANTHROPIC_API_KEY`	Railway	La llave de la API de Anthropic (56). **Nunca** en un archivo ni en el chat.
`LINKEDIN_AGENT_MODE=propose`	Railway	Enciende las propuestas diarias (56). Sin definir = `off` = igual que hoy.
`LINKEDIN_AGENT_DAILY_BUDGET_USD`	Railway	Tope de gasto diario (D1; por defecto 1.00).
`LINKEDIN_AGENT_MODEL`, `ANTHROPIC_PRICE_IN_PER_MTOK`, `ANTHROPIC_PRICE_OUT_PER_MTOK`	Railway	Modelo y precios para calcular el costo.
`OG_SHOW_LOGOS=on`	Vercel (comunidad)	**Dejar apagado** hasta resolver la licencia de logos (D3).
`NEXT_PUBLIC_SITE_URL`	Vercel (comunidad)	Ya existe (por defecto `https://avotalent.app`); las URL de imagen OG se arman con ella.

Cada vez que se prende o apaga uno: `npm run log-change -- "…" --kind config --area <tarea> --expects "<métrica>" --at <hora UTC con Z>`.

C. Lista de archivos nuevos o tocados por tarea

Tarea	Backend	Comunidad / panel / agente
21	`sql/link-check-migration.sql`, `services/vacancies/link-check.ts`, `scripts/link-check.ts`, `services/cron.service.ts`, ruta `/links` en `analytics.routes.ts`, `analytics-access.middleware.ts`, `tests/unit/link-check.test.ts`	panel `oferta/page.tsx` (tarjeta), `mcp-analytics/src/panel.ts` (`vacancy_links`)

22	`sql/apply-people-migration.sql`, `services/analytics/apply-people.ts`, ruta `/web/apply-people`, middleware, tests (SQL con PGlite)	panel `trafico/postulacion/page.tsx`, `panel.ts` (`web_apply_people`)
56	`sql/linkedin-agent-migration.sql`, `services/linkedin/agent/*`, `vacancy-publisher.ts` (gancho), `cron.service.ts`, `alerts.ts`, rutas `/linkedin-proposals…`, `package.json` (`@anthropic-ai/sdk`), tests	panel `linkedin-propuestas/page.tsx` + `NAV_GROUPS`
32	`seo.routes.ts` (`/seo/company/:slug`), `vacancy-post.ts` (`cardFor`)	`app/og/**/route.tsx`, `lib/og-card.tsx`, `lib/og.ts`, `app/layout.tsx`, `(main)/page.tsx`, layouts de vacante y empresa, listados, `tests/og-metadata.test.ts`
45	`sql/profile-set-migration.sql`, `people-value.ts`, `engagement-history.ts`, ruta `/people-value` y `/summary`, tests	panel `personas/page.tsx`, `panel.ts` (descripción de `people_value`)

D. Antes de cada `git push` (no hay staging)

1. `cd backend && npx tsc --noEmit && npx vitest run` — todo verde (hay una prueba, `api-timing`, que a veces falla bajo carga y pasa sola al correrla aparte; no es de estas tareas).
2. `cd apps/community && npx tsc --noEmit && npm run test` y `cd apps/avocado && npx tsc --noEmit`; `cd packages/mcp-analytics && npx tsc --noEmit && npx vitest run`.
3. Commit con línea `Expects: …` en el cuerpo (`feat` / `fix` / `perf`) y la viñeta nueva en `CLAUDE.md`.
4. Subir **una tarea por vez** (un push por tarea) para poder ver en `get_change_log` y en los registros de Railway cuál causó algo.
5. Después del despliegue: comprobar que el servidor arrancó sin errores y que la ruta nueva responde (en **«no está lista»** si la migración no se ha corrido, nunca 500).

E. Lo que Francisco devuelve al terminar cada tarea

- **Qué cambió realmente** respecto a esta guía (nombres, rutas, campos) — para que el jefe de producto actualice su documento.
- **Cómo se probó**, con qué casos (los de cada tarea) y el resultado de `vitest`.
- **Qué quedó fuera y por qué.**
- **Hora de publicación en UTC** para anotarla en `get_change_log`, y los valores de las medidas de referencia que obtuvo (p. ej., los conteos de la tarea 45).
- Un mensaje corto al fundador con **sus pasos**: qué SQL pegar, qué variable poner, uno por uno.

Un recordatorio sobre la muestra. La medición empezó el 30 oct 2026 y la comunidad es de ≈100 cuentas: todo antes/después es indicativo, no prueba de causa. El 32 y el 56 miden LinkedIn en las mismas semanas y no se podrán separar: se anotan las fechas y se lee a 14 días como correlación.